

Metodi per la valutazione dell'efficacia delle librerie di Input Validation

Confronto tra la libreria ESAPI e Hibernate Validator Framework

1. Introduzione

1.1 Obiettivi:

Rendere sicura una Web Application costruita su un Microservizio, sul backend. Illustrando nei vari aspetti del confronto, quali vantaggi e svantaggi offrano la libreria ESAPI e Hibernate Validator Framework.

1.2 Panoramica Attacchi:

Gli attacchi di Injection per realizzare il confronto sono i seguenti:

SQL Injection e Cross Site Scripting XSS.

SQL Injection

Il SQL Injection è un tipo di attacco incentrato sulla estrapolazione delle informazioni dal database relazionale SQL. L'attacco opera nel seguente modo:

L'attaccante cerca di modificare il comportamento della Query SQL alterando il contenuto delle informazioni utilizzate sotto forma di stringa che vengono concatenate a quest'ultima.

Il risultato è che se queste sottostringhe contengono commenti, istruzioni SQL, è possibile riuscire ottenere informazioni dal database relazionale SQL. Ci sono diversi modi per proteggere una Web Application:

- Evitare di concatenare le sottostringhe contenenti le informazioni per la Query SQL, ma utilizzare gli appositi metodi forniti dalla libreria che effettua le interrogazioni al DB.
- Verificare il contenuto delle sottostringhe con delle espressioni regolari.

- I valori numerici utilizzare le variabili numeriche anziché le stringhe alfanumeriche, così da limitare drasticamente i caratteri accettati.
- Le Query SQL che visualizzano i dati dal DB utilizzare l'istruzione LIMIT mantenendo un numero di record basso come ad esempio 25. Questo nel caso in cui l'attacco avviene con successo limiterebbe un po' i danni.
- Aggiungere le sequenze di Escape per i rispettivi caratteri.

FONTE: OWASP TOP TEN

Cross Site Scripting (XSS)

Il Cross Site Scripting (XSS) è un tipo di attacco che si concentra nell'estrarre informazioni dalle vittime usando il blocco di codice dannoso.

Il Cross Site Scripting (XSS) ha due modalità di attacco:

Reflected: Questo tipo di attacco è non persistente. Esso si realizza inserendo il blocco di codice dannoso in un sito web vulnerabile che verrà eseguito dalla vittima attraverso una fonte esterna. La vittima crederà di navigare su un sito web sicuro, poiché anche il Browser Web è ingannato, ma in realtà senza saperlo ha ricevuto questo blocco di codice dannoso. Come fonte esterna si intende, ad esempio, il collegamento ipertestuale inviato tramite Email alla vittima.

Stored: Questo tipo di attacco è persistente. Esso si realizza inserendo dentro ad un form di una pagina del sito web il codice dannoso, il quale lo memorizzerà nel suo database. Quando le vittime navigheranno sul sito web vulnerabile, le informazioni estrapolate dal database contengono anche il blocco di codice dannoso che può essere eseguito dal Browser Web delle vittime.

Ci sono vari modi per proteggere una Web Application:

- Verificare il contenuto dei parametri di URL o di un URI.
- Sanificare le stringhe contenenti i marcatori HTML.
- Verificare che le stringhe alfanumeriche inserite dentro al database siano pulite e non contengano i marcatori HTML con codice dannoso.
- Realizzare un output encoding tale da non fare eseguire il codice dannoso.

FONTE: OWASP TOP TEN

1.3 Security by Design:

Lo scopo del Security by Design è quello di soddisfare i requisiti di sicurezza durante il ciclo di sviluppo di un software. Garantendo, sin da subito, uno standard di sicurezza che evita potenziali danni agli utenti, riducendo i costi e i tempi di sviluppo.

Il Security by Design prevede diversi principi fondamentali che devono essere rispettati durante il ciclo di sviluppo.

1. Il principio del privilegio minimo e della separazione dei compiti.
2. Il principio della difesa in profondità.
3. Il principio di fiducia zero.
4. Il principio di sicurezza nell'open source.

Il principio del privilegio minimo e della separazione dei compiti:

Il principio del privilegio minimo assicura che gli utenti dovrebbero avere il minimo accesso necessario per svolgere il proprio lavoro. Questo significa che gli utenti dovrebbero avere il giusto accesso alle sole risorse di cui hanno bisogno per portare a termine il loro lavoro.

Il principio della separazione dei compiti assicura che nessun individuo abbia il controllo di tutti gli aspetti su tutti gli aspetti di una operazione all'interno di un sito web o di un software. Questo è realizzato assegnando compiti diversi a persone diverse. Quindi si riduce il rischio di frodi ed errori, oltre a garantire una certa velocità nel completare le attività.

Il principio della difesa in profondità:

Il principio della difesa in profondità assicura una strategia con più livelli di sicurezza per proteggere le risorse di una organizzazione. Si basa sul fatto che se uno dei livelli di sicurezza fallisce gli altri garantiscono la protezione della risorsa. I livelli inclusi in questo principio sono i seguenti: Sicurezza fisica, sicurezza di rete, sicurezza delle applicazioni e dei dati. Il fine è creare un ambiente sicuro che sia resiliente e in grado di rilevare gli attacchi col fine di rispondere tempestivamente agli incidenti di sicurezza.

Il principio di fiducia zero:

Questo principio presuppone che tutti gli utenti, dispositivi e reti non siano attendibili e quindi come conseguenza devono essere verificati prima che venga

concesso l'accesso alle risorse di una organizzazione. Tale verifica non si limita alla sola autenticazione e autorizzazione, ma anche ad un costante monitoraggio e controllo delle attività svolte dagli utenti. Garantendo così l'accesso solo a coloro che ne hanno effettivamente bisogno, specialmente ai dati sensibili. Il fine è ridurre il rischio di violazione dei dati e altri incidenti di sicurezza.

Il principio di sicurezza nell'open source:

Questo principio sottolinea l'importanza della sicurezza nello sviluppo di software open source. Esso garantisce che gli sviluppatori siano consapevoli delle implicazioni sulla sicurezza del loro codice sorgente e quindi decidano di adottare le misure necessarie per garantirne la sicurezza. Questo prevede pratiche di codifiche sicure, test delle vulnerabilità, l'uso di strumenti di sviluppo sicuri.

FONTE: OWASP TOP TEN

1.4 La strategia di Input Validation:

La strategia di input validation prevede principalmente la verifica del contenuto dei dati da validare mediante l'uso delle espressioni regolari, impostate in modo tale che rispettino l'impostazione di Whitelist. Quindi consentire solo il contenuto con il quale non è possibile effettuare danni alla Web Application. Quando l'espressione regolare applicata alla stringa da validare restituisce un esito positivo, quindi c'è una corrispondenza, allora la stringa non contiene sottostringhe che possono fare danni alla Web Application. Altrimenti, il programmatore dovrà scegliere se bloccare la richiesta http e non portarla al termine, oppure sanificare la stringa con ulteriori espressioni regolari.

1.5 Criteri di confronto:

I paragrafi dedicati al confronto per la libreria ESAPI e il Framework Hibernate Validator, presenti all'interno del capitolo sugli elementi di valutazione, hanno per ciascuno i seguenti paragrafi:

- **Performance:** Questo paragrafo illustra l'organizzazione dei tempi medi di esecuzione e deviazioni standard espressi in millisecondi all'interno di una tabella. Realizzando per ciascuna richiesta HTTP programmata all'interno della Web Application tre righe nella tabella, contenenti i tempi raccolti mediante i test su JMeter.
- **Completezza:** Questo paragrafo illustra l'organizzazione delle validazioni realizzate all'interno della Web Application. Illustrando anche eventuali alternative e le criticità della libreria o del framework all'interno delle due

Web Application con eventuali soluzioni adottate. Inoltre, è presente una tabella contenente per ciascuna richiesta HTTP, un giudizio che determina l'esito dei tentativi di attacchi svolti. È presente anche una leggenda che spiega i giudizi presenti nella tabella.

- **Flessibilità:** Questo paragrafo illustra gli aspetti legati alla comodità di utilizzo della libreria ESAPI e del Framework Hibernate Validator e alla documentazione. Nello specifico, sono argomentati i seguenti aspetti:
 - La libreria o il framework consentono di creare delle classi personalizzate per effettuare la validazione.
 - Quante righe di codice sono necessarie per utilizzare le validazioni standard presenti nella libreria o nel framework.
 - In che modo è possibile personalizzare il comportamento delle validazioni.
 - Quante righe di codice sono necessarie per utilizzare le validazioni personalizzate.
 - Come è organizzata la documentazione e quali difficoltà si sono presentati per trovarla.
 - Il codice implementato funziona oppure ha presentato qualche problema nel suo funzionamento.
- **Robustezza:** Questo paragrafo illustra gli aspetti dedicati alla gestione degli errori e l'impatto delle possibili falle di sicurezza. Nello specifico, gli aspetti argomentati sono i seguenti:
 - Come la libreria o framework gestisce gli errori di validazione. Inoltre, se la presenza di questi errori interrompono l'esecuzione della Web Application.
 - Come la libreria o framework gestisce gli errori sulle modalità di validazione configurati in modo errato dal programmatore. Ad esempio, una espressione regolare con errori di sintassi. Inoltre, se la presenza di questi errori interrompono la compilazione dei sorgenti o l'esecuzione della Web Application.
 - La libreria o il framework in che modalità permette al programmatore di personalizzare una possibile configurazione che ne determina il comportamento e se presente quest'ultima è errata o insicura in che modo è gestita.
 - Illustrare eventuali problemi di sicurezza presenti nella libreria o nel framework e in che modo questi possono mettere a rischio la Web Application.

2. Descrizione della libreria ESAPI ed il Framework Hibernate Validator

Libreria ESAPI

ESAPI è una libreria che effettua controlli per la sicurezza delle web application scritte in Java. Sviluppata da OWASP e rilasciato sotto licenze: BSD e Creative Commons.

Questa libreria facilita il programmatore nello scrivere applicazioni a basso rischio, ed è stata progettata per semplificare l'aggiunta successiva dei controlli di sicurezza nelle applicazioni esistenti. Inoltre, fornisce un insieme di interfacce per il controllo della sicurezza. La logica implementata nella libreria ESAPI non è specifica sulle organizzazioni e nemmeno sulle applicazioni.

I metodi utilizzati nel progetto sono i seguenti:

- **canonicalize()**: Semplifica la codifica della stringa. Questo metodo è contenuto nella classe Encoder che deve essere istanziata.
- **isValidInput()**: Permette di effettuare la validazione, ed utilizza come parametri le stringhe dedicate al contesto, la stringa da validare e il nome della espressione regolare. L'espressione regolare deve essere presente nei file properties di ESAPI. Questo metodo ritorna un valore booleano ed è contenuto nella classe DefaultValidator che deve essere istanziata. Inoltre, effettua una chiamata a "canonicalize" internamente come impostazione predefinita.
- **encodeForSQL()**: Questo metodo in base al Codec utilizzato, si occupa di inserire le sequenze di escape nelle stringhe prima che queste vengano concatenate nella stringa della query SQL. Questo metodo è contenuto nella classe Encoder che deve essere istanziata.
- **isValidSafeHTML()**: Questo metodo effettua la validazione di una stringa che contiene i marcatori HTML 5 e si assicura che non sia presente codice JavaScript con il quale è possibile effettuare un attacco di Cross Site Scripting. Questo metodo richiede come parametri:
 - La stringa di contesto.
 - La stringa da validare.

- La lunghezza massima.
- Un valore booleano per specificare se acconsentire le stringhe vuote.
- Un oggetto della classe `ValidationErrorMessageList` che contiene la lista di errori di validazione.

Questo metodo ritorna un valore booleano come esito della validazione, ma effettua anche una chiamata al metodo `getValidSafeHTML` che a sua volta effettua una chiamata al metodo `canonicalize`. Esso è contenuto nella classe `DefaultValidator` che deve essere istanziata. Infine, come requisito per il funzionamento è richiesto il file `antisamy-esapi.xml` che deve essere presente nella cartella `resources` del rispettivo progetto Maven.

- **isValidDate():** Questo metodo effettua la validazione di una stringa che contiene data e ora, tra i suoi parametri sono presenti la stringa di contesto, la stringa dell'input da validare, una istanza della classe `DateFormat` che contiene il formato della data, la variabile booleana per consentire stringhe da validare vuote. Esso ritorna un valore booleano come esito della validazione, in caso di errore genera l'eccezione `IntrusionException`, ed è contenuto nella classe `DefaultValidator`.
- **isValidDouble():** Questo metodo effettua la validazione di un `Double` come stringa, fornendogli minimo e massimo valore come `Double`. Esso ritorna un valore booleano come esito della validazione, ed è contenuto nella classe `DefaultValidator`.
- **isValidInteger():** Questo metodo effettua la validazione di un `Integer` come stringa, fornendogli minimo e massimo valore come `Integer`. Esso ritorna un valore booleano come esito della validazione, ed è contenuto nella classe `DefaultValidator`.
- **isValidURI():** Questo metodo effettua la validazione di un URI passato come stringa, specificando con un valore booleano se la stringa può essere vuota. Esso ritorna un valore booleano come esito della validazione, ed è contenuto nella classe `DefaultValidator`.

Hibernate Validator Framework

Questo Framework consente di impostare vincoli su determinate informazioni all'interno di una applicazione per validarne il contenuto. Questo Framework è sviluppato da Hibernate ed è rilasciato sotto licenza Apache 2.0. I vincoli si impostano mediante l'uso delle annotazioni in Java, ad esempio:

//Questa annotazione stabilisce che la stringa deve avere una lunghezza compresa

//tra 2 e 10 caratteri alfanumerici.

@Size(min = 2, max = 10)

//Questa annotazione stabilisce che la stringa deve rispettare la seguente

//espressione regolare

@Pattern(regex = "[A-Za-z0-9]*", message="Username is not valid")

private String username;

Bisogna osservare che i vincoli non solo è possibile impostarne più di uno su un elemento da validare, ma è possibile anche creare vincoli personalizzati.

Inoltre, ha un insieme di classi e interfacce con i rispettivi attributi e metodi per effettuare la validazione dei dati. I metodi utilizzati per effettuare la validazione sono i seguenti:

- `validate()`: Questo metodo prende in esame un intero oggetto istanziato e ritorna gli errori in un insieme che contiene le istanze della classe parametrica `ConstraintViolation`, dove come parametro deve essere specificata la classe dell'oggetto da validare. Questo metodo appartiene all'interfaccia `Validator` che deve essere opportunamente istanziata dalla classe `ValidatorFactory`.
- `validateProperty()`: Questo metodo effettua la validazione di una singola proprietà dell'oggetto istanziato preso in esame, specificando come stringa il nome della proprietà da validare. Anch'esso ritorna gli errori in un insieme che contiene le istanze della classe parametrica `ConstraintViolation`. Questo metodo appartiene all'interfaccia `Validator` che deve essere opportunamente istanziata dalla classe `ValidatorFactory`.

3. Ambiente di test ed Elementi di valutazione

3.1 Ambiente di test

Microservizio Spring Boot

Spring Boot consente di realizzare delle Web Application utilizzando l'architettura di microservizi. Permettendo di avere un ambiente dove è necessaria una minima configurazione. Inoltre, utilizzando le varie librerie di Spring è possibile ottenere una grande flessibilità nello scrivere il codice sorgente. Oltre ad avere una documentazione e delle guide semplici per realizzare le varie funzionalità della Web Application. Per l'ambiente di test è stato utilizzato Spring Boot per realizzare due web application. Una per realizzare i test sulla libreria ESAPI, l'altra per realizzare i

test sul Framework Hibernate Validator. Le due web application sono scritte in Java 11, utilizzano due progetti Maven e Apache Tomcat Web Server con le seguenti librerie di Spring:

- Spring Data JPA: Necessaria per realizzare le interrogazioni sul rispettivo database relazionale SQL.
- MySQL Driver: Necessaria per realizzare la connessione e le interrogazioni sul rispettivo database MySQL.
- Spring Data MongoDB: Necessaria per realizzare la connessione e le interrogazioni sul rispettivo database non relazionale MongoDB.
- Spring Web: Necessaria per realizzare i REST Controller e le altre componenti del Design Pattern MVC.
- Thymeleaf: Un Template Engine Server Side necessario per realizzare le View.
- Spring Security: Necessaria per realizzare i meccanismi di autenticazione e controllo degli accessi.

Inoltre, sono presenti ulteriori librerie non appartenenti a Spring Boot:

- Log4J: Necessaria per realizzare i log all'interno delle due web application.
- Lombok: Necessaria per realizzare i metodi costruttori e i metodi Getter e Setter più rapidamente all'interno delle classi.
- Jackson Databind: Necessaria per realizzare il disaccoppiamento delle classi che contengono le informazioni sulla struttura del database e le interrogazioni, da quelle che contengono solo le informazioni estrapolate dal database relazionale.
- Springdoc OpenAPI UI: Contiene SwaggerUI, ed è necessaria per ottenere una pagina HTML dove è possibile visualizzare ed utilizzare le richieste http programmate nei Rest Controller.

Tutte queste librerie sono state utilizzate per realizzare due web application che condividono le stesse funzionalità. Le funzionalità sono le seguenti:

- Un database relazionale MySQL che contiene le informazioni degli utenti e sulle note che scrivono.
- Un database non relazionale MongoDB che contiene le informazioni sugli errori di validazione.
- Un sistema di autenticazione degli utenti registrati nel database relazionale. Il sistema di accesso utilizza i cookie per mantenere la sessione attiva.
- Un Logger per registrare i log sulla console delle due web application.
- Una pagina HTML dove è possibile visualizzare e provare le richieste http programmate nei REST Controller delle due web application.

- Una libreria (Jackson Databind) per realizzare il disaccoppiamento delle classi che contengono le informazioni sulla struttura del database e le interrogazioni, da quelle che contengono solo le informazioni estrapolate dal database relazionale.
- Una libreria (Lombok) per velocizzare la realizzazione delle classi contenenti le informazioni e la struttura sul database.

Le due web application hanno due REST Controller dedicati alle operazioni sui dati degli utenti e delle loro note:

- Utenti:
 - Registrazione
 - Aggiornamenti delle informazioni
 - Ricerca per Username
 - Ricerca per Email
 - Cancellazione
- Note:
 - Salvataggio
 - Aggiornamento delle note
 - Ricerca per Username
 - Ricerca per Email
 - Cancellazione

Ogni singola operazione programmata nei due REST Controller effettua le opportune validazioni usando le rispettive funzionalità di ESAPI e Hibernate prima di effettuare le operazioni sul rispettivo database relazionale. Nel caso in cui si verificano errori di validazione, questi vengono opportunamente registrati sul rispettivo database non relazionale. Infine, sugli esiti delle validazioni e delle operazioni sul database descritte precedentemente, viene generata la risposta col rispettivo messaggio da inviare al Client HTTP.

- Esito positivo:
 - Codice HTTP:
 - 201 Creazione
 - 202 Accettazione
 - Stringa di caratteri alfanumerici
 - Lista di record in formato JSON
- Esito negativo:
 - Codice HTTP:
 - 400 Cattiva richiesta

- 403 Proibito
- 500 Errore interno del server
- Stringa di caratteri alfanumerici
- Elenco di errori in formato JSON

Il disaccoppiamento è stato realizzato tra le classi del REST Controller e le classi che interagiscono direttamente col Database. La descrizione successiva descrive come è stato realizzato il disaccoppiamento per il REST Controller dedicato agli utenti, ma l'impostazione è la medesima con qualunque REST Controller.

Si crea un livello di astrazione che separa il REST Controller dedicato agli utenti dallo UserRepository e da User, introducendo le classi UserService e UserDTO.

Il REST Controller utilizza UserService e UserDTO per interagire col database. Tuttavia, UserRepository e User rimangono le classi principali per tale interazione, queste ultime sono utilizzate da UserService. In UserService per realizzare la conversione da User e UserDTO sono stati implementati due metodi privati per realizzare una conversione bidirezionale, che utilizzano classi e metodi della libreria Jackson Databind. I metodi privati di conversione tra User e UserDTO operano nel seguente modo:

La classe SimpleBeanPropertyFilter è stata utilizzata per poter creare un filtro che serializza tutti gli attributi di una classe ad eccezione dell'ID, utilizzando il metodo serializeAllExcept. Successivamente con la classe FilterProvider si crea un insieme di filtri nel quale si aggiunge il filtro di serializzazione sulle proprietà ad eccezione dell'ID, utilizzando il metodo setFilterProvider.

Utilizzando l'oggetto istanziato della classe ObjectMapper si imposta il suo insieme di filtri e viene effettuata la conversione tra le due classi User e UserDTO (la conversione avviene in una sola direzione), utilizzando rispettivamente i metodi setFilterProvider e convertValue. Infine, utilizzando gli Optional controllo se l'ID dell'oggetto da convertire esiste, nel caso affermativo questo viene inserito nell'oggetto convertito ottenuto precedentemente dal metodo convertValue.

Affinché tutto funzioni correttamente, la classe User deve implementare l'interfaccia Serializable.

Gli altri metodi di UserService sono quelli che interagiscono con i metodi di UserRepository e questi utilizzano anche i due metodi privati di conversione.

Il sistema di autenticazione degli utenti registrati nel database relazionale MySQL, utilizza la libreria Spring Security. Questo sistema di accesso genera un cookie chiamato JSESSIONID, utile per testare la validazione centralizzata dei cookie.

Il sistema di autenticazione è stato realizzato partendo dalla creazione della classe che utilizza l'annotazione `EnableWebSecurity`. Questa classe ha al suo interno il metodo di `securityFilterChain` che contiene l'oggetto `http` della classe `HttpSecurity`. Questo oggetto definisce il comportamento che le varie pagine e richieste `http` programmate nei `REST Controller` devono assumere con il sistema di accesso. Ovvero, tutte le pagine e richieste `HTTP` ad eccezione della registrazione, login, homepage richiedono tutte che l'utente sia autenticato. Altrimenti in caso di utente non autenticato viene generato il reindirizzamento alla pagina di login.

Tuttavia, è stata disabilitata la protezione contro gli attacchi di `Cross Site Request Forgery` dal momento che viene utilizzato `Postman` e `JMeter` per interagire con i `REST Controller`. Poiché se lasciato abilitato è richiesto ad ogni richiesta `http` il parametro `_csrf` con un valore che viene generato solo dal server, ciò implicava che per ogni richiesta `http` ci fosse una pagina `HTML` con un form.

Le pagine `HTML` presenti nelle due `Web Application` sono le seguenti:

- Il login con il form che utilizza il metodo `http` di `post`.
- La pagina di homepage.
- Una pagina `hello`.

È necessario impostare anche il metodo `encoder` che contiene e ritorna il `PasswordEncoder` da utilizzare per codificare le password. Queste password codificate sono confrontate con quelle presenti nella tabella utenti del database relazionale.

È stata scelta la classe `BCryptPasswordEncoder` come `PasswordEncoder` per le password nella classe `WebSecurityConfig` nel metodo `encoder`.

Le informazioni dell'utente sono state specificate in due classi:

- La prima `UserDTODetails` che implementa l'interfaccia `UserDetails` con i rispettivi metodi implementati ed utilizza l'oggetto `UserDTO`.
- La seconda `UserDTODetailsService` che invece effettua l'interrogazione al DB usando `UserService` e `UserDTO`.

L'organizzazione del contenuto realizzato in queste classi serve per mantenere l'impostazione per il disaccoppiamento, descritto precedentemente.

Infine, tra le funzionalità utilizzate di Spring all'interno della `Web Application` dedicata alla libreria `ESAPI`, sono stati impostati un `Interceptor` e un `Filter` dedicati alla validazione centralizzata. L'`Interceptor` è uno strumento che permette di intercettare tutte le richieste `http` di tutti i `REST Controller`, grazie ai suoi tre metodi:

- `preHandle()`: Questo metodo viene eseguito prima dell'arrivo della richiesta al rispettivo metodo del REST Controller. Infine, ritorna un valore booleano che permette la scelta di continuare la normale esecuzione della richiesta HTTP oppure interrompere con messaggio personalizzato.
- `postHandle()`: Questo metodo viene eseguito dopo l'arrivo della richiesta al rispettivo metodo del REST Controller. Quindi subito dopo l'esecuzione di quest'ultimo.
- `afterCompletion()`: Per svolgere ulteriori operazioni dopo il completamento della richiesta HTTP e dopo aver generato una potenziale view.

Il metodo dell'interceptor utilizzato per le validazioni centralizzate è il `preHandle()`.

Affinché tutto funzioni è necessario creare delle classi per gli interceptor che implementino l'interfaccia `HandlerInterceptor` e bisogna implementare i tre metodi citati precedentemente. Inoltre, è necessario creare una classe di configurazione che implementi l'interfaccia `WebMvcConfigurer` e implementare il metodo `addInterceptors`, nel quale si aggiungono le istanze degli interceptor. Infine, bisogna aggiungere anche la annotazione del Framework Spring, ovvero `Configuration` sulla classe dedicata alla configurazione.

Il Filter invece è uno strumento che lavora nella libreria Spring Security, per utilizzarlo è necessario creare una classe che estende la sopra classe `GenericFilterBean`. Nel momento in cui estendiamo questa classe è necessario implementare il metodo `doFilter`. Una volta implementato questo metodo è possibile realizzare il blocco di istruzioni al suo interno, che eseguirà il filter ogni volta che viene richiamato. In questo filter specifico per il progetto che utilizza la libreria ESAPI, è stato necessario implementare il codice per effettuare la validazione centralizzata della richiesta di login con metodo http di POST. Poiché questa richiesta http non veniva intercettata dall'interceptor. Al suo interno è stato istanziato l'oggetto della classe `UserControllerHandler` che viene utilizzato dall'interceptor e utilizza il metodo `handler`. Questo metodo restituisce un valore booleano e nel caso sia impostato su false, viene lanciata una eccezione che blocca la normale esecuzione della richiesta HTTP di login, poiché viene generato un messaggio d'errore con codice HTTP 403. Inoltre, i parametri formali che usano le classi `ServletRequest` e `ServletResponse` possono essere convertiti (quindi soggetti ad una operazione di cast) rispettivamente in `HttpServletRequest` e `HttpServletResponse`.

Una volta creata la classe dedicata al filter è necessario specificare nel metodo `securityFilterChain` della classe che utilizza l'annotazione `EnableWebSecurity`, come deve essere utilizzato il filter. Ci sono tre metodi nella classe `HttpSecurity` per definire tale utilizzo:

- `addFilterBefore()`: Questo metodo assicura che il filter venga attivato prima di un filter già implementato da Spring Security. Specificando l'istanza della classe del filter personalizzato e il nome della classe del filter già implementato da Spring Security.
- `addFilterAfter()`: Questo metodo assicura che il filter venga attivato dopo un filter già implementato da Spring Security. Specificando l'istanza della classe del filter personalizzato e il nome della classe del filter già implementato da Spring Security.
- `addFilter()`: Questo metodo assicura che il filter venga aggiunto, tuttavia il suddetto filter deve estendere uno dei filter forniti da Spring Security.

Per il filter appena creato è stato utilizzato il metodo `addFilterBefore` per definire il suo comportamento ed è attivato prima di `UsernamePasswordAuthenticationFilter`, che è un filtro già implementato da Spring Security.

JMeter

Questo programma scritto da Apache Software Foundation e sotto licenza Apache License 2.0, permette di programmare dei test automatizzati sulle due web application. Lo scopo di questi test è determinare la velocità della libreria ESAPI e del Framework Hibernate Validator. Il primo modello di test configurato utilizza:

- Un Thread Group per impostare il numero dei thread e il numero di iterazioni che devono eseguire.
- Una HTTP Request che è l'operazione che devono eseguire i thread.
- Un grafico per i risultati.

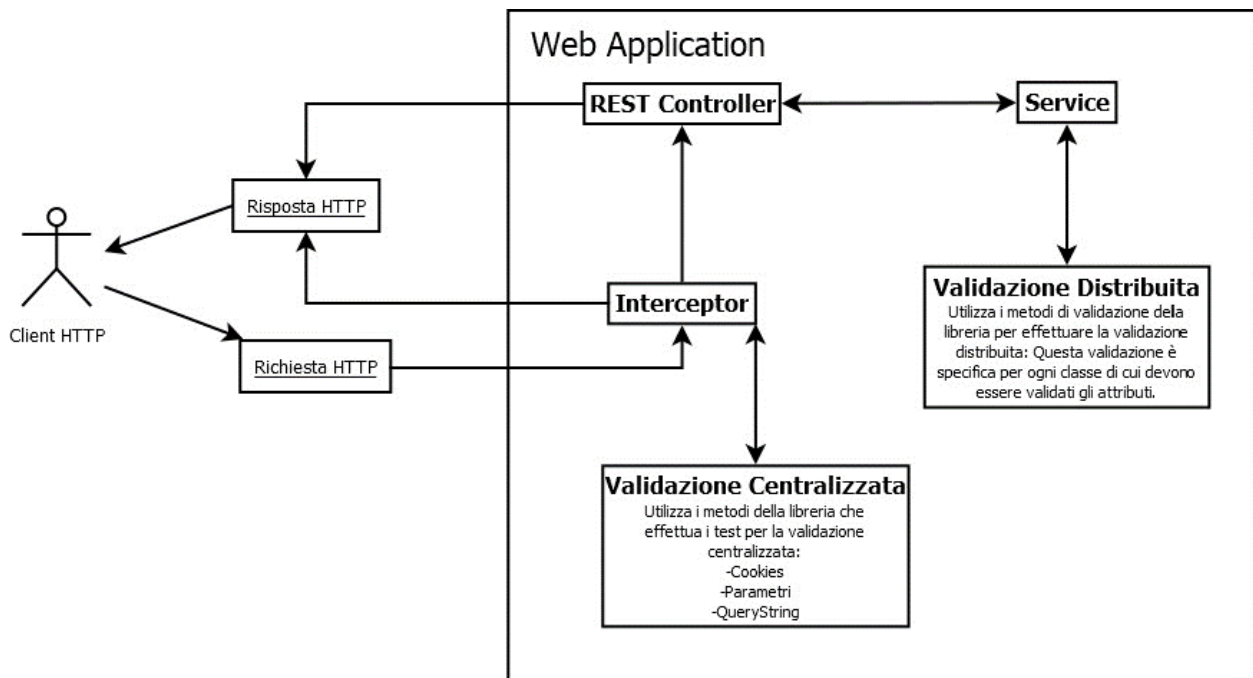
Il grafico mostra sull'asse delle ascisse le operazioni e sull'asse delle ordinate i millisecondi. Inoltre, mostra le linee delle informazioni, media, mediano, deviazione, throughput. Sotto al grafico sono mostrati: Il numero dei campioni (il numero di operazioni), media, deviazione, throughput, mediano. Ogni singolo test utilizza 10 thread, ognuno dei quali eseguono 150 test. Infine, solo ed esclusivamente per JMeter nelle due web application è stato temporaneamente disabilitato l'accesso dell'utente obbligatorio.

Postman

Postman è un software che consente di testare le richieste http programmate nei REST Controller di una Web Application. Se registriamo un account è possibile salvare tutte richieste http desiderate, dare loro un nome e organizzarle. Questo software ha una licenza proprietaria, ma alcune componenti sono open source. Postman è stato utilizzato per verificare il corretto funzionamento delle due Web

Application durante la realizzazione delle nuove funzionalità. Ma anche per verificare se gli attacchi di SQL Injection e Cross Site Scripting venivano rilevati correttamente.

Schema che illustra il funzionamento delle richieste http all'interno delle due Web Application



Il client http manda la sua richiesta alla Web Application. La Web Application riceve la richiesta, la quale viene intercettata dall'interceptor. Questa richiesta viene validata con la validazione centralizzata e se il contenuto risulta pulito, il metodo del REST Controller effettuerà la validazione distribuita. Servendosi di un metodo di una classe di servizio che effettuerà la validazione in base all'oggetto istanziato da validare, di conseguenza, richiama il metodo di validazione della rispettiva classe di validazione. Se non ci sono errori di validazione allora la Web Application eseguirà l'operazione contenuta nella richiesta e fornirà la rispettiva risposta al client http. Tuttavia, in caso di errore, in base alla validazione che ha restituito tale errore, viene realizzata una risposta specifica per il client http.

3.2 Elementi di valutazione

Performance Libreria ESAPI

Le performance di questa libreria sono state determinate mediante i test realizzati su JMeter. Questi test utilizzano il Thread Group che lancia 10 thread e per ognuno di essi vengono eseguite 150 richieste HTTP come operazioni, per un totale di 1500 richieste. I test svolti sulla Web Application in totale sono 33 e sono suddivisi in tre gruppi da 11 test ciascuno:

- I test svolti con i dati puliti, quindi nessun tentativo di attacchi SQL Injection e Cross Site Scripting.
- I test svolti con i dati predisposti per effettuare tentativi di attacchi SQL Injection.
- I test svolti con i dati predisposti per effettuare tentativi di attacchi Cross Site Scripting.

I test contengono i seguenti dati che sono espressi in millisecondi ad eccezione del Throughput, quest'ultimo è espresso per operazioni al minuto. Il suo valore è inversamente proporzionale sui tempi delle operazioni. I dati contenuti nei test sono stati organizzati nelle seguenti tabelle:

Dati puliti						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	59	95,7	43,31	5,925.341	76
/user/register	1500	64	156,95	73,28	3,677.973	177
/user/update	1500	65	125,65	77,17	4,541.097	91
/user/findByEmail	1500	5	6,31	3,53	46,948.357	5
/user/findByUsername	1500	2	4,47	1,96	59,016.393	4
/user/delete	1500	4	8,87	3,49	42,412.818	8
/note/save	1500	10	13,57	6,24	29,930.163	12
/note/update	1500	12	21,08	17,38	18,614.271	15
/note/findByEmail	1500	101	46,86	40,83	10,818.608	34
/note/findByUsername	1500	2	34,13	33,53	13,771.997	22
/note/delete	1500	4	6,96	2,71	48,231.511	6
SQL Injection						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	3	5,83	2,51	49,073.064	5
/user/register	1500	4	8,15	3,21	39,045.553	8
/user/update	1500	13	14,78	9,17	26,216.137	11
/user/findByEmail	1500	2	4,26	1,41	57,251.908	4
/user/findByUsername	1500	3	3,09	1,1	65,454.545	3
/user/delete	1500	2	3,51	1,23	67,567.568	4
/note/save	1500	5	7,98	2,85	41,997.2	7
/note/update	1500	4	8,17	6,04	39,284.155	8
/note/findByEmail	1500	2	2,47	0,8	69,390.902	2
/note/findByUsername	1500	2	2,9	1,03	58,365.759	4
/note/delete	1500	3	4,42	1,15	56,179.775	4
Cross Site Scripting						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	6	6,35	3,08	46,177.527	6
/user/register	1500	12	15,18	9,7	27,863.777	12
/user/update	1500	5	14,35	8,03	27,889.681	12

/user/findByEmail	1500	2	3,24	1,33	66,964.286	3
/user/findByUsername	1500	1	2,99	1,24	67,466.267	3
/user/delete	1500	1	2,79	1,11	70,038.911	3
/note/save	1500	4	8,33	2,23	41,493.776	8
/note/update	1500	4	9,66	3,24	37,484.382	9
/note/findByEmail	1500	3	3,02	1,66	65,885.798	3
/note/findByUsername	1500	2	2,72	1,1	68,223.51	3
/note/delete	1500	2	3,3	1,17	62,981.106	3

Completezza Libreria ESAPI

La libreria ESAPI, dal punto di vista della completezza offre la possibilità a chi la utilizza di avere un insieme notevole di funzionalità che gli permettono di implementare la sicurezza di una Web Application a più livelli, rendendo quest'ultima resiliente. Ma tuttavia, seppur molto completa dal punto di vista delle funzionalità i problemi non mancano. Nel progetto dedicato alla libreria ESAPI, questa libreria lavora su due modalità di validazione dei dati:

- **Validazione centralizzata:** La validazione centralizzata si occupa di validare tutti i parametri, QueryString, cookies di ogni singola richiesta http affinché non contengano irregolarità dal punto di vista della sicurezza.
- **Validazione distribuita:** La validazione distribuita si occupa di validare i dati ottenuti da parametri e JSON in modo specifico per ogni richiesta http affinché i dati inseriti nel database siano corretti sia dal punto di vista del contenuto che dal punto di vista della sicurezza.

La libreria ESAPI per effettuare la validazione centralizzata mette a disposizione al programmatore due modalità per effettuare le validazioni:

- Validazione con metodo specifico di una specifica classe.
- Validazione generale con due metodi di due specifiche classi.

La validazione con un metodo specifico nel caso di validazione centralizzata prevede l'uso della classe DefaultHTTPUtilities con i rispettivi metodi per le rispettive componenti:

- **Parametri:** `getParameter(HttpServletRequest request, String name)` questo metodo ritorna un parametro validato correttamente, utilizzando l'espressione regolare definita dentro al file ESAPI.properties.
- **Cookie:** `getCookie(HttpServletRequest request, String name)` questo metodo ritorna un cookie validato correttamente, utilizzando l'espressione regolare definita dentro al file ESAPI.properties.

- **QueryString:** Non esiste nessun metodo specifico per la sua validazione, ma esiste l'espressione regolare definita dentro al file ESAPI.properties.

La validazione generale è stata realizzata con i due metodi di due specifiche classi. Questa validazione usa le Wrapper Function dei metodi canonicalize e isValidInput, già citati precedentemente nella descrizione della libreria ESAPI. Questi due metodi sono quelli effettivamente utilizzati nel progetto, poiché isValidInput ritorna un booleano che permette una gestione semplificata del controllo in caso di errore, all'interno della Wrapper Function. La Wrapper Function in caso di errore registrerà nel rispettivo database MongoDB l'errore di validazione. Le validazioni per parametri e cookies non hanno presentato problemi particolari, mentre quella per la QueryString ha presentato alcuni problemi. Bisogna tenere presente che isValidInput al suo interno come comportamento predefinito effettua una chiamata al metodo canonicalize, che nel caso della QueryString se non rileva la codifica mischiata (Mixed Encoding), può potenzialmente cambiare il carattere ampersand in base alla sequenza di caratteri che si presentano nel nome del parametro successivo. Ad esempio, se il parametro successivo è chiamato "note", il metodo canonicalize vede la sequenza "...¬..." come una codifica e sostituisce questi quattro caratteri in un singolo carattere. Una possibile soluzione per aggirare tali problemi legati alla validazione della QueryString è la seguente: Effettuare prima la validazione dei parametri, sia per nome che per contenuto. Successivamente, se nessun errore è stato riscontrato in tutte le validazioni dei parametri, si valida la QueryString. Tuttavia, in caso di errori generati dal metodo canonicalize è possibile ignorarli nel caso in cui i parametri sono stati validati tutti correttamente. Altrimenti, consideriamo anche questo ultimo errore di validazione. Tuttavia, con questo particolare problema e con questa possibile soluzione è stato necessario implementare le seguenti due librerie di Apache Commons:

- httpclient-4.3.3
- httpcore-4.3.3

Queste due librerie permettono l'uso delle seguenti due classi:

- URIBuilder: Questa classe gestisce le singole parti di un URI (Uniform Resource Identifier).
- URLEncodedUtils: Questa classe gestisce i parametri codificati e assicura la codifica UTF-8 col metodo parse.

La validazione centralizzata nel progetto dedicato alla libreria ESAPI è stata realizzata mediante l'uso dell'interceptor e del filter. La validazione dei Cookie invece ha presentato una limitazione legata ai caratteri accettati per la validazione del loro

valore. La limitazione riguarda l'espressione regolare HTTPCookieValue presente nel file ESAPI.properties. Questa espressione regolare accetta solo i seguenti caratteri:

- Caratteri alfanumerici dello standard US ASCII
- Trattino (Score)
- Barra (Forward Slash)
- Simbolo del più
- Simbolo uguale
- Trattino basso (Underscore)
- Spazio

L'ultimo standard del Cookie descritto nel documento RFC 6265 presente nel sito della IETF afferma che i caratteri consentiti sono i seguenti:

- Caratteri standard dello US ASCII ad esclusione di quelli di controllo (da 0 a 31 e 127 in decimale)
- Spazio
- Doppie virgolette
- Punto e virgola
- Barra rovesciata (Backslash)

La validazione distribuita nel progetto dedicato alla libreria ESAPI è stata realizzata nei metodi dei rispettivi REST Controller ad eccezione della richiesta http di login che viene effettuata dentro al Filter. Essi richiamano i metodi di validazione dalle loro rispettive classi di validazione. In queste classi, per ogni singola componente di cui deve essere validato il contenuto, sono presenti le chiamate alle Wrapper Function dei metodi: canonicalize, encodeForSQL, isValidInput, isValidSafeHTML, isValidDate, isValidDouble, isValidInteger, isValidURI. Tuttavia, isValidInput non utilizzerà sempre le espressioni regolari originali presenti nel file validation.properties. Ma all'interno di questo file sono state aggiunte ulteriori espressioni regolari personalizzate per la validazione del contenuto di questi dati. Ad esempio, tra i dati della tabella dell'utente è presente il codice fiscale che richiede una espressione regolare apposita. Le espressioni regolari fornite con la libreria ESAPI non sono soggette agli attacchi Cross Site Scripting, ma con gli attacchi SQL Injection qualche attacco passa con successo, specialmente se sono presenti i Forward Slash nei valori dei parametri. Il metodo encodeForSQL sulle stringhe delle email è presente il problema che compaiono le sequenze di escape sui record del database una volta memorizzate. Quindi è meglio memorizzare le stringhe delle email con la codifica semplificata.

Le espressioni regolari fornite dalla libreria ESAPI impiegate nella validazione centralizzata sono le seguenti:

- Nome del Cookie: HTTPCookieName
- Valore del Cookie: HTTPCookieValue
- Protocollo Http: HTTPScheme
- Nome del server Http: HTTPServerName
- Percorso Http: HTTPServletPath
- Nome del Parametro: HTTPParameterName
- Valore del Parametro: HTTPParameterValue
- QueryString: HTTPQueryString

Le espressioni regolari fornite dalla libreria ESAPI impiegate nella validazione distribuita sono le seguenti:

- String sicura: SafeString
- Email: Email

Tabella contenente gli esiti degli attacchi SQL Injection e Cross Site Scripting:

Metodo	Richiesta	Esito SQL Injection	Esito Cross Site Scripting
POST	/login	Ok ma attenzione	OK
POST	/user/register	Ok ma attenzione	OK
POST	/user/update	Ok ma attenzione	OK
GET	/user/findByEmail	Ok ma attenzione	OK
GET	/user/findByUsername	Ok ma attenzione	OK
DELETE	/user/delete	OK	OK
POST	/note/save	Ok ma attenzione	OK
POST	/note/update	Ok ma attenzione	OK
GET	/note/findByEmail	Ok ma attenzione	OK
GET	/note/findByUsername	Ok ma attenzione	OK
DELETE	/note/delete	OK	OK

Leggenda tabella:

- **OK**: Gli attacchi sono stati rilevati e annullati correttamente dalla validazione centralizzata.
- **Ok ma attenzione**: Gli attacchi sono stati rilevati e annullati in parte dalla validazione centralizzata e l'altra parte dalla validazione distribuita.
- **Attenzione**: Gli attacchi sono stati rilevati e annullati correttamente dalla validazione distribuita.

- **Non sicuro**: Gli attacchi hanno superato entrambe le validazioni.

Tutti i test di validazione hanno dato esito positivo. Tuttavia, dove è riportato “Ok ma attenzione” si intende che alcune validazioni centralizzate non sono andate a buon fine, quindi un livello di sicurezza è stato superato da un attacco. Bisogna tenere presente che l’impostazione della validazione centralizzata e della validazione distribuita permette una gestione resiliente degli attacchi.

Flessibilità Libreria ESAPI

La libreria ESAPI sulla flessibilità permette al programmatore di implementare i controlli di sicurezza a più livelli, anche scrivendo delle classi apposite. Tuttavia, non permette la validazione distribuita di interi oggetti che contengono più attributi da validare. Quindi è necessario scrivere delle classi di validazioni per questi oggetti che in base al numero di attributi da validare, può essere necessario scrivere classi di una certa dimensione. La validazione di un singolo attributo di una classe prevede un minimo di quattro righe. La validazione personalizzata nella libreria ESAPI è gestita grazie ai suoi file properties e file xml che permettono di aggiungere espressioni regolari personalizzate per svolgere tale compito. Quindi gli strumenti per la validazione forniti da ESAPI sono configurabili e personalizzabili mediante questi file properties e file xml.

La documentazione della libreria ESAPI si trova su internet e sul sito ufficiale di OWASP forniscono:

- Javadoc che fornisce anche esempi di codice nei commenti. Ma non è l’ideale per chi ha bisogno di una guida per iniziare.
- Repository su GitHub.
- Ebook in PDF scaricabile, anche se un po’ datato dal momento che risale al 2008 e non si trova nella pagina della libreria. Quindi per trovarlo bisogna usare un motore di ricerca.

L’implementazione del codice ha presentato qualche difficoltà, in particolar modo il metodo canonicalize che è estremamente sensibile alle codifiche mischiate. Quindi se sono presenti stringhe lunghe e complesse, questo metodo crea delle difficoltà in termini di falsi positivi oppure sostituisce alcuni caratteri importanti che non devono essere sostituiti. Infine, la libreria ESAPI permette di fare un late adoption in un progetto esistente con il vantaggio che si possono creare delle Wrapper Function mantenendo il codice del progetto più pulito.

Robustezza Libreria ESAPI

La libreria ESAPI fornisce gli errori attraverso i log, inoltre, nonostante gli errori di validazione continua a funzionare. Tuttavia, sempre sulla gestione degli errori, dal momento che all'avvio della Web Application carica i file properties contenenti configurazione ed espressioni regolari, nel caso questi file non sono presenti, l'esecuzione di tutta la Web Application viene interrotta. Invece, se ad esempio una delle espressioni regolari personalizzate presentano errori di sintassi, questi sono mostrati nei log della console della Web Application, ma non comporta alcuna interruzione. I metodi utilizzati nel progetto per la validazione nel caso presentano errori, ritornano il valore booleano false. Questo permette al programmatore di poter utilizzare tale valore booleano per realizzare dei blocchi di istruzione alternativi per gestire tali situazioni. La libreria ESAPI, tuttavia, ha due falle di sicurezza aperte come mostrato sul Repository di GitHub nella apposita sezione chiamata "Security". Una di queste falle di sicurezza è chiamata "Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') in org.owasp.esapi:esapi – antisamy-esapi.xml configuration file" e consente di effettuare attacchi di tipo Cross Site Scripting. Questo attacco può essere realizzato mediante l'uso dei due metodi chiamati isValidSafeHTML e getValidSafeHTML, che a loro volta utilizzano le espressioni regolari presenti nel file "antisamy-esapi.xml". Le seguenti espressioni regolari presentano i problemi di sicurezza: onsiteURL, htmlTitle, offsiteURL. Questa falla di sicurezza ha anche il suo CVE ID è "CVE-2022-24891", ed ha una gravità moderata. Nonostante sia ancora visibile sulla sezione della sicurezza nel Repository di GitHub, è stato specificato anche l'intervallo di versioni della libreria ESAPI soggette a potenziali attacchi di Cross Site Scripting. Le versioni affette sono quelle precedenti alla 2.3.0.0. Viceversa, dalla versione 2.3.0.0 e successive, la falla di sicurezza non rappresenta più un problema poiché è stata sistemata. La versione della libreria ESAPI utilizzata nella Web Application è la 2.5.2.0.

Performance Hibernate Validator Framework

Le performance di questo Framework sono state determinate mediante i test realizzati su JMeter. Questi test utilizzano il Thread Group che lancia 10 thread e per ognuno di essi vengono eseguite 150 richieste HTTP come operazioni, per un totale di 1500 richieste. I test svolti sulla Web Application in totale sono 33 e sono suddivisi in tre gruppi, che contengono 11 test ciascuno:

- Gli 11 test svolti con i dati puliti, quindi nessun tentativo di attacchi SQL Injection e Cross Site Scripting.
- I 11 test svolti con i dati predisposti per effettuare tentativi di attacchi SQL Injection.
- I 11 test svolti con i dati predisposti per effettuare tentativi di attacchi Cross Site Scripting.

I test contengono i seguenti dati che sono espressi in millisecondi ad eccezione del Throughput, quest'ultimo è espresso per operazioni al minuto. Il suo valore è inversamente proporzionale sui tempi delle operazioni. I dati contenuti nei test sono stati organizzati nelle seguenti tabelle:

Dati puliti						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	66	110,136	58,33	5,211.651	80
/user/register	1500	63	173,68	52,87	3,328.895	185
/user/update	1500	66	103,09	46,07	5,499.542	78
/user/findByEmail	1500	4	4,38	1,73	52,356.021	4
/user/findByUsername	1500	3	3,87	1,21	56.250	4
/user/delete	1500	14	8,42	3,44	42,134.831	8
/note/save	1500	3	9,33	6,6	38,200.34	8
/note/update	1500	8	8,2	3,64	452.786	7
/note/findByEmail	1500	3	6,44	3,27	45,045.045	6
/note/findByUsername	1500	2	5,77	3,96	51,724.138	5
/note/delete	1500	3	7,04	2,57	45,754.957	6
SQL Injection						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	3	3,85	2,15	60,200.669	3
/user/register	1500	3	6,41	3,59	48,727.666	5
/user/update	1500	10	9,85	6,3	33,733.133	9
/user/findByEmail	1500	2	2,46	0,98	69,444.444	2
/user/findByUsername	1500	2	3,18	1,09	62,717.77	3
/user/delete	1500	2	1,89	0,94	76,530.612	2
/note/save	1500	6	6,31	3,26	46,487.603	6
/note/update	1500	3	5,27	1,57	49,972.238	5
/note/findByEmail	1500	2	2,72	0,98	67,064.083	3
/note/findByUsername	1500	1	2,27	1,22	72,289.157	2
/note/delete	1500	0	1,6	0,74	82,644.628	2
Cross Site Scripting						
HTTP Request	Samples	Latest Sample	Average	Deviation	Throughput	Median
/login	1500	5	7,14	3,51	42,095.416	6
/user/register	1500	3	4,45	1,9	54,911.531	4
/user/update	1500	3	10,1	5,59	36,348.95	9

/user/findByEmail	1500	3	3,77	1,19	55,180.871	4
/user/findByUsername	1500	2	2,59	1,01	63,335.679	3
/user/delete	1500	5	4,82	2,18	55,418.719	4
/note/save	1500	2	4,4	1,34	56,426.332	4
/note/update	1500	7	6,67	3,23	39,700.044	7
/note/findByEmail	1500	4	3,76	1,62	56,962.025	4
/note/findByUsername	1500	2	2,36	1,45	70,422.535	2
/note/delete	1500	1	1,6	0,82	77,452.668	1

Completezza Hibernate Validator Framework

Hibernate Validator Framework fornisce a chi la utilizza un insieme di strumenti col fine di realizzare la validazione per contenuto dei dati, prima che questi vengano inseriti in un database di una Web Application. Nel progetto dedicato a Hibernate Validator Framework, questo Framework è stato impiegato per realizzare la validazione distribuita dei dati. Tuttavia, con i suoi strumenti di base questa la libreria non fornisce alcuna forma di sicurezza contro SQL Injection e Cross Site Scripting. Però fornisce al programmatore un vincolo utile che può utilizzare per implementare la sicurezza nella validazione distribuita. Il vincolo è chiamato Pattern e permette di applicare delle espressioni regolari sulle stringhe col fine di garantire non solo la validazione per contenuto, ma anche la validazione dal punto di vista della sicurezza. Nel progetto realizzato tutte le stringhe validate con Hibernate Validator Framework utilizzano questo vincolo. Altrimenti senza questo vincolo la Web Application è soggetta a SQL Injection e Cross Site Scripting. Nella validazione distribuita, questa viene realizzata all'interno dei metodi dei rispettivi REST Controller dopo l'operazione di Unmarshaling, utilizzando i metodi citati precedentemente nella descrizione del Framework. Inoltre, questi metodi forniscono con un messaggio l'errore di validazione. Questi messaggi è possibile raccogliarli in un oggetto e spedirlo al rispettivo client http, inoltre, è possibile anche memorizzarli su un database non relazionale creando delle Wrapper Function sui metodi di validazione. Affinché funzioni, è necessario assicurarsi che i dati da validare nelle rispettive classi abbiano i rispettivi vincoli dichiarati utilizzando le annotazioni Java. Inoltre, una volta inizializzato un insieme di istanze della classe parametrica ConstraintViolation con una specifica classe, non è possibile utilizzare tale insieme con una classe differente da quella dichiarata. Nonostante i vincoli di base del Framework non siano sicuri, c'è il vincolo Email che nella maggior parte dei casi rileva i tentativi di SQL Injection, ma non in tutti i casi. Nello specifico, non rileva i tentativi fatti utilizzando Forward Slash. Infine, per validare il testo delle note è stato realizzato un vincolo di validazione personalizzato che ha una sua annotazione "MySafeHTML". Questo vincolo è

composto da una annotazione chiamata `MySafeHTML` e da una classe chiamata `MyHTMLValidator`. All'interno della annotazione è stato specificato il messaggio d'errore predefinito ed i due metodi `groups` e `payload`. Inoltre, la annotazione utilizza le seguenti annotazioni Java di Hibernate:

- **Target:** Dove è specificato l'obiettivo, in questo caso il campo, che è specificato dal valore `FIELD` appartenente a `ElementType`, il quale è un enumeratore.
- **Retention:** Questo specifica la politica di conservazione, in questo caso a `Runtime`, che è specificato dal valore `RUNTIME` appartenente a `RetentionPolicy`, il quale è un enumeratore.
- **Constraint:** Questo specifica che è un vincolo e che deve essere validato da una classe, la quale in questo caso è `MyHTMLValidator`.

La classe `MyHTMLValidator` implementa l'interfaccia parametrica `ConstraintValidator`, la quale utilizza come parametri la annotazione `MySafeHTML` e la classe `String`. Questa interfaccia parametrica richiede che siano implementati due metodi `initialize` e `isValid`. Il secondo metodo è quello dove viene definito il comportamento del vincolo di validazione. In questo caso è stato richiamato il metodo `isValid` e ritornato il suo valore booleano, questo metodo appartiene alla libreria `Jsoup`. Esso richiede come argomenti la stringa da validare e di specificare la `Whitelist` di marcatori HTML consentiti. `Jsoup` fornisce una classe per la `Whitelist` di marcatori HTML chiamata `Safelist` e al suo interno sono disponibili diversi metodi per ottenere la lista di marcatori predefiniti:

- **`none()`:** Questo metodo consente solo i text nodes.
- **`simpleText()`:** Questo metodo consente solo del testo semplice con i seguenti marcatori HTML 5: `b`, `em`, `i`, `strong`, `u`.
- **`basic()`:** Questo metodo consente i seguenti marcatori HTML 5: `a`, `b`, `blockquote`, `br`, `cite`, `code`, `dd`, `dl`, `dt`, `em`, `i`, `li`, `ol`, `p`, `pre`, `q`, `small`, `span`, `strike`, `strong`, `sub`, `sup`, `u`, `ul`. Inoltre, per i collegamenti ipertestuali possono essere di tipo `http`, `https`, `ftp`, `mailto` e possono avere applicato l'attributo `rel=nofollow`.
- **`basicWithImages()`:** Questo metodo a differenza del metodo `basic` consente anche l'uso del marcatore HTML 5 per le immagini `img` con gli attributi appropriati e i tipi collegamenti consentiti sono solo `http` e `https`.
- **`relaxed()`:** Questo metodo consente l'uso dei seguenti marcatori HTML 5: `a`, `b`, `blockquote`, `br`, `caption`, `cite`, `code`, `col`, `colgroup`, `dd`, `div`, `dl`, `dt`, `em`, `h1`, `h2`, `h3`, `h4`, `h5`, `h6`, `i`, `img`, `li`, `ol`, `p`, `pre`, `q`, `small`, `span`, `strike`, `strong`, `sub`, `sup`, `table`, `tbody`, `td`, `tfoot`, `th`, `thead`, `tr`, `u`, `ul`.

Infine, la classe Safelist fornisce anche i metodi per personalizzare i marcatori HTML 5 permessi.

Tabella contenente gli esiti degli attacchi SQL Injection e Cross Site Scripting:

Metodo	Richiesta	Esito SQL Injection	Esito Cross Site Scripting
POST	/login	OK	OK
POST	/user/register	OK	OK
POST	/user/update	OK	OK
GET	/user/findByEmail	OK	OK
GET	/user/findByUsername	OK	OK
DELETE	/user/delete	OK	OK
POST	/note/save	OK	OK
POST	/note/update	OK	OK
GET	/note/findByEmail	OK	OK
GET	/note/findByUsername	OK	OK
DELETE	/note/delete	OK	OK

Leggenda tabella:

- **OK**: Gli attacchi sono stati rilevati e annullati correttamente dalla validazione distribuita.
- **Non sicuro**: Gli attacchi hanno superato la validazione distribuita.

Tutte le validazioni effettuate ad eccezione delle due richieste di cancellazione per l'utente e per le note sono state svolte da Hibernate Validator Framework con la modalità della validazione distribuita. Tuttavia, non è possibile realizzare la validazione del login perché la logica è gestita internamente da Spring Security. L'unico modo di effettuare tale validazione è attraverso il Filter che è stato già predisposto svolgere tale compito, ciò richiederebbe la creazione di un oggetto della classe UserDTO nella quale sono presenti Username e Password che verrebbero validati singolarmente. Questa validazione corrisponderebbe alla validazione distribuita, poiché specifica per il login.

Flessibilità Hibernate Validator Framework

Hibernate Validator Framework permette al programmatore una gran flessibilità, infatti gli permette sia di realizzare delle classi apposite per la validazione di interi oggetti contenenti gli attributi con i loro rispettivi vincoli per la validazione, sia di evitare di scrivere tali classi e di realizzare con pochissime righe di codice la validazione, nello specifico sono necessarie quattro righe di codice per effettuare la validazione di un oggetto che ha i vincoli di validazioni definiti al suo interno. Inoltre,

qualora non sia necessario validare tutto l'oggetto, ma solo un suo attributo, Hibernate fornisce un metodo specifico anche questo. Questo Framework consente anche al programmatore di creare delle classi e annotazioni che contengono e gestiscono dei vincoli personalizzati, queste classi in base alle necessità del programmatore possono essere piccole oppure grandi. La quantità minima di righe di codice necessario per realizzare un vincolo personalizzato è di 18 righe e comprende sia l'annotazione che la classe. Tuttavia, per facilitare la manutenzione è preferibile mantenere le suddette classi piccole e semplici con poche responsabilità (funzionalità). La guida ufficiale per iniziare di Hibernate Validator Framework al capitolo 6 fornisce una guida dettagliata su come realizzare un vincolo personalizzato, fornendo anche un esempio. La documentazione ufficiale di Hibernate Validator Framework si trova sul sito ufficiale e fornisce non solo la documentazione per l'ultima versione, ma mantiene le documentazioni anche per le precedenti versioni. Queste documentazioni sono disponibili in vari formati:

- HTML
- PDF
- Javadoc

Inoltre, la documentazione fornisce i seguenti manuali:

- Reference (HTML, PDF, Javadoc)
- Guida ufficiale per iniziare (HTML, PDF)
- Guida per la migrazione (HTML)

La guida ufficiale per iniziare è ideale per chi deve utilizzare questo Framework per la prima volta, poiché include anche le istruzioni per aggiungere la libreria al proprio progetto Maven. Inoltre, entrambi i formati disponibili di questa guida contengono al loro interno un indice per muoversi tra i vari capitoli.

Infine, Hibernate permette la possibilità di fare un late adoption in un progetto esistente anche se le modifiche sono più invasive dal momento che richiede l'inserimento dei vincoli di validazione proprio sugli elementi da validare.

Robustezza Hibernate Validator Framework

Hibernate Validator Framework sulla gestione degli errori dal momento che utilizza un insieme per le violazioni dei vincoli, questi sono registrati con una specifica stringa che indica l'errore e l'elemento non validato correttamente. Questo permette al programmatore di restituire in modo specifico l'errore al rispettivo client http. Tuttavia, questi errori non sono registrati nella console attraverso i log. Quindi sarà compito del programmatore assicurarsi che questa caratteristica sia presente.

Inoltre, non avendo file properties specifici da caricare all'avvio della Web Application, garantisce che la Web Application possa inicializzarsi correttamente senza mancanze da parte del programmatore. Hibernate Validator Framework nelle sue versioni precedenti alla 6.0.18 aveva un vincolo di validazione chiamato SafeHtml che permetteva la validazione di stringhe contenenti i marcatori HTML 5. Questo vincolo di validazione aveva una falla di sicurezza che è stata opportunamente segnalata ed il suo CVE ID è il seguente: CVE-2019-10219. Inoltre, presentava una gravità moderata che permetteva l'esecuzione di codice dannoso per realizzare attacchi di Cross Site Scripting. Questa falla di sicurezza ha costretto gli sviluppatori di Hibernate a rendere questo vincolo di validazione deprecato. Quindi è necessario scrivere un vincolo di validazione personalizzato per consentire l'uso dei marcatori HTML all'interno dei testi delle note.

4. Conclusioni

Il confronto tra la Libreria ESAPI e Hibernate Validator Framework si conclude che la Libreria ESAPI fornisce un insieme di strumenti migliori per la sicurezza. Tuttavia, è meno flessibile e meno documentata di Hibernate Validator Framework. Sulle performance Hibernate è estremamente veloce, anche se in alcuni casi la differenza è di pochi millisecondi, tranne per quanto riguarda la registrazione, login, cancellazione delle note. Mentre la cancellazione degli utenti solo nel caso del SQL Injection è più lento. Sulla robustezza hanno modalità simili per gestire gli errori di validazione e gli errori di configurazione delle validazioni. Entrambe hanno avuto falle di sicurezza che permettevano gli attacchi di Cross Site Scripting che sono state risolte. Ma su Hibernate hanno deprecato il vincolo di validazione coinvolto. Invece, sulle performance Hibernate è più veloce di ESAPI anche perché il progetto dedicato a Hibernate non ha la validazione centralizzata. Infine, per il reale utilizzo di ESAPI e Hibernate, bisogna valutare in base al progetto quale delle due utilizzare. Tenendo presente se il progetto viene fatto da zero oppure è già esistente. I vari paragrafi affrontati nel capitolo "Elementi di valutazione" sono riassunti mediante le seguenti tabelle:

Completezza:

Elemento	ESAPI	Hibernate
Validazione centralizzata		
Cookies	Restrittivo	Assente

Parameters	Attenzione	Assente
QueryString	Crea problemi	Assente
Validazione distribuita		
Parameters	OK	OK
JSON	OK	OK

Legenda tabella completezza:

- OK: Ben gestito con eventuali alternative.
- Attenzione: Non sempre ben gestito.
- Crea problemi: La funzionalità è presente, ma crea diversi problemi per utilizzarlo.
- Assente: Non gestito.
- Realizzabile: Non gestito direttamente dalla libreria o framework ma è realizzabile da parte del programmatore.
- Restrittivo: La validazione è restrittiva e non consente tutti i caratteri presenti nello standard come consentiti.

La validazione centralizzata su ESAPI è presente, ma presenta problemi.

La validazione dei cookies è restrittiva, quella dei parametri lascia passare determinate stringhe di SQL Injection, quella del QueryString causa falsi positivi e modifica le stringhe. Mentre su Hibernate la validazione centralizzata è totalmente assente, ma il programmatore può con un po' di lavoro creare queste validazioni. La validazione distribuita invece è gestita da entrambi.

Flessibilità:

Elemento	ESAPI	Hibernate
Personalizzazione Validazione		
Classi personalizzate per validazione	Realizzabile	Realizzabile
Quantità righe di codice per validazioni standard	Minimo quattro righe per la validazione di un singolo attributo di una classe.	Minimo quattro righe per la validazione sia di un singolo attributo sia per validare un intero oggetto.
Quantità righe di codice per validazioni personalizzate	Non realizzabile tramite codice, ma tramite espressioni regolari	Minimo 18 righe
Configurare comportamento dei metodi di validazione	Realizzabile attraverso i file properties e xml	Realizzabile attraverso i parametri delle annotazioni dei vincoli di validazione
Documentazione		
Qualità della documentazione	Discreta qualità	Ottima qualità
Organizzazione della documentazione	Discreta organizzazione	Ottima organizzazione
Difficoltà nel trovare la documentazione	Ricerca difficile	Ricerca facile
Codice implementato		
Difficoltà nel codice implementato	Implementazione normale	Implementazione facile
Il Late Adoption è realizzabile	Realizzabile, anche creando una libreria di Wrapper Function	Realizzabile, ma richiede una modifica più invasiva del codice sorgente, poiché i vincoli devono essere inseriti nelle



classi contenenti i dati da
validare

Legenda tabella flessibilità:

Validazioni:

- Realizzabile: La funzionalità è realizzabile.
- Non realizzabile: La funzionalità non è realizzabile.

Documentazione:

- Ottima qualità: La documentazione è di ottima qualità.
- Discreta qualità: La documentazione è di una qualità discreta.
- Pessima qualità: La documentazione è di una qualità che rende difficile al programmatore utilizzare la libreria o il framework.
- Ottima organizzazione: La documentazione ha un'ottima organizzazione.
- Discreta organizzazione: La documentazione ha una discreta organizzazione.
- Pessima organizzazione: Il programmatore è spesso disorientato e impiega tempo nello sviluppare la sua Web Application.
- Ricerca facile: La documentazione si trova per intero sul sito ufficiale.
- Ricerca difficile: La documentazione si trova in diversi punti del sito web, ciò ha richiesto l'utilizzo di un motore di ricerca.

Codice implementato:

- Implementazione facile: Il codice implementato non ha creato difficoltà.
- Implementazione normale: Il codice implementato ha creato qualche difficoltà che ha richiesto qualche soluzione per superare i problemi.
- Implementazione complicata: Il codice implementato ha richiesto diverse soluzioni per garantirne il funzionamento.

Sulla personalizzazione delle validazioni, Hibernate dimostra di essere più personalizzabile di ESAPI, poiché permette la creazione di validazioni custom. La documentazione che offre Hibernate è migliore perchè disponibile in diversi tipi e formati, inoltre la mantiene per le vecchie versioni del framework.

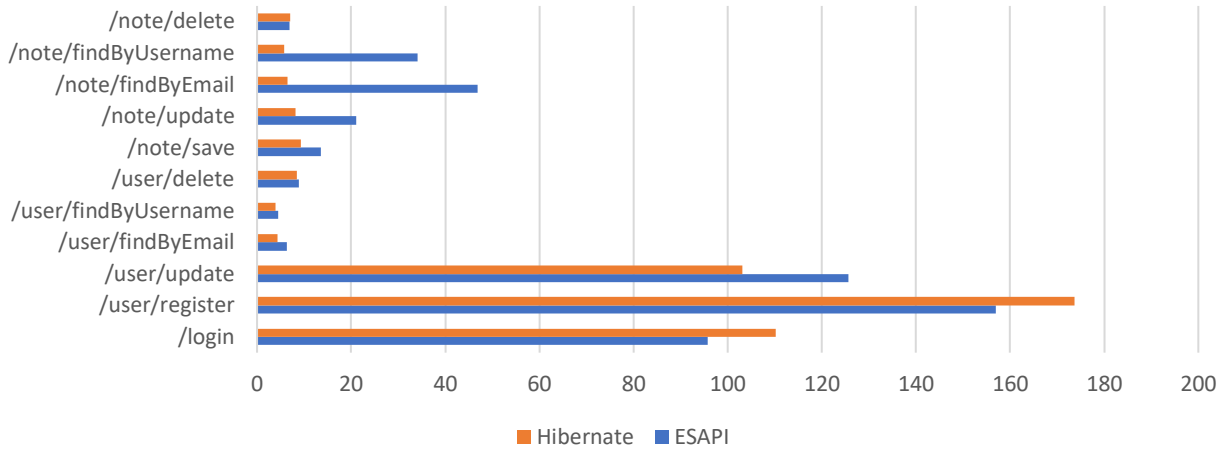
ESAPI invece ha solo il Javadoc. Sul codice implementato, ESAPI ha creato qualche difficoltà. Invece, sulla possibilità di realizzare un late adoption, entrambi lo permettono con la differenza che Hibernate richiede una modifica più invasiva del codice.

Perfomance:

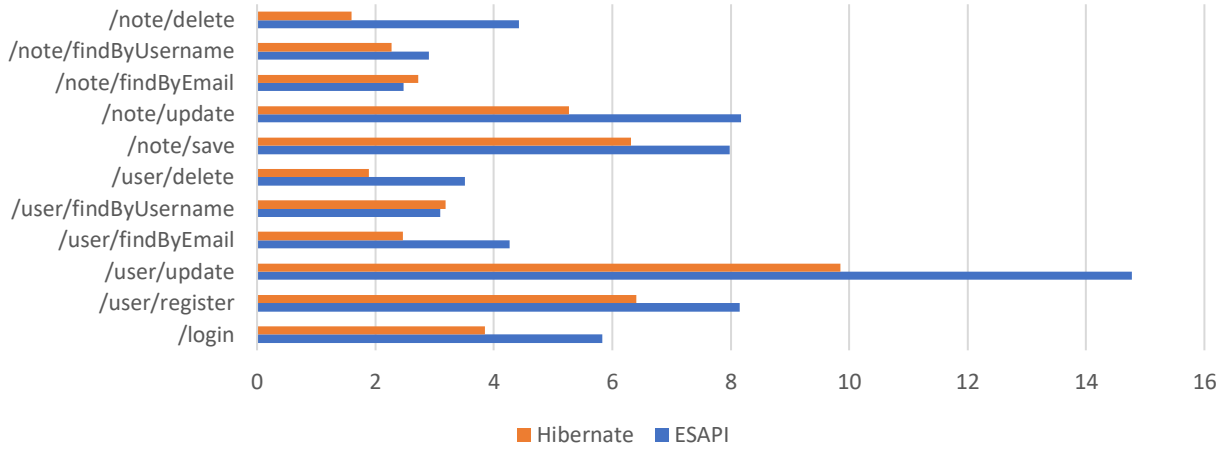
Dati puliti	Media		Deviazione Standard	
HTTP Request	ESAPI	Hibernate	ESAPI	Hibernate
/login	95,7	110,136	43,31	58,33
/user/register	156,95	173,68	73,28	52,87
/user/update	125,65	103,09	77,17	46,07
/user/findByEmail	6,31	4,38	3,53	1,73
/user/findByUsername	4,47	3,87	1,96	1,21
/user/delete	8,87	8,42	3,49	3,44
/note/save	13,57	9,33	6,24	6,6
/note/update	21,08	8,2	17,38	3,64
/note/findByEmail	46,86	6,44	40,83	3,27
/note/findByUsername	34,13	5,77	33,53	3,96
/note/delete	6,96	7,04	2,71	2,57
SQL Injection	Media		Deviazione Standard	
HTTP Request	ESAPI	Hibernate	ESAPI	Hibernate
/login	5,83	3,85	2,51	2,15
/user/register	8,15	6,41	3,21	3,59
/user/update	14,78	9,85	9,17	6,3
/user/findByEmail	4,26	2,46	1,41	0,98
/user/findByUsername	3,09	3,18	1,1	1,09
/user/delete	3,51	1,89	1,23	0,94
/note/save	7,98	6,31	2,85	3,26
/note/update	8,17	5,27	6,04	1,57
/note/findByEmail	2,47	2,72	0,8	0,98
/note/findByUsername	2,9	2,27	1,03	1,22
/note/delete	4,42	1,6	1,15	0,74
Cross Site Scripting	Media		Deviazione Standard	
HTTP Request	ESAPI	Hibernate	ESAPI	Hibernate
/login	6,35	7,14	3,08	3,51
/user/register	15,18	4,45	9,7	1,9
/user/update	14,35	10,1	8,03	5,59
/user/findByEmail	3,24	3,77	1,33	1,19
/user/findByUsername	2,99	2,59	1,24	1,01
/user/delete	2,79	4,82	1,11	2,18
/note/save	8,33	4,4	2,23	1,34
/note/update	9,66	6,67	3,24	3,23
/note/findByEmail	3,02	3,76	1,66	1,62
/note/findByUsername	2,72	2,36	1,1	1,45
/note/delete	3,3	1,6	1,17	0,82

I dati dei tempi medi e deviazioni standard sono riportati in millisecondi.

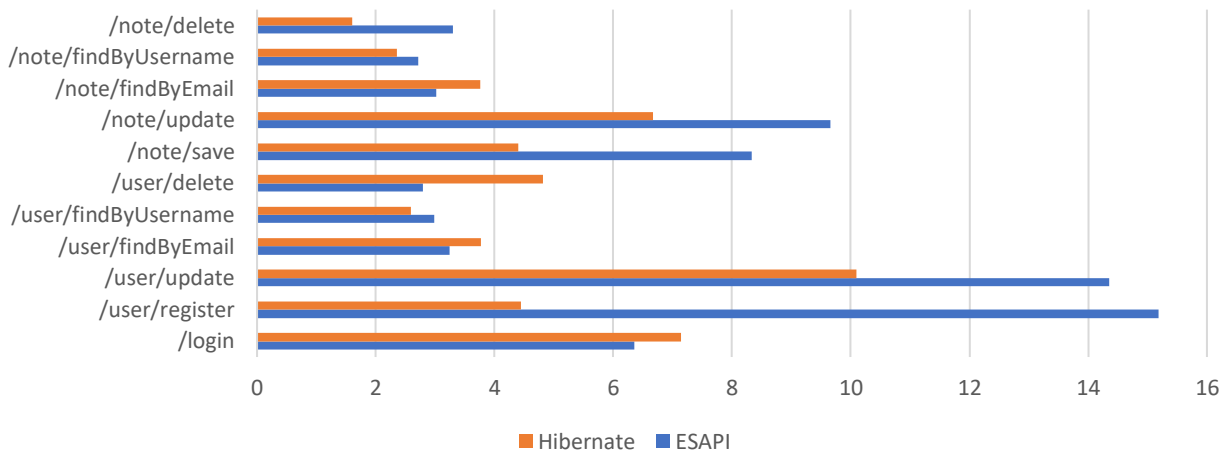
Tempi medi - Dati puliti (millisecondi)



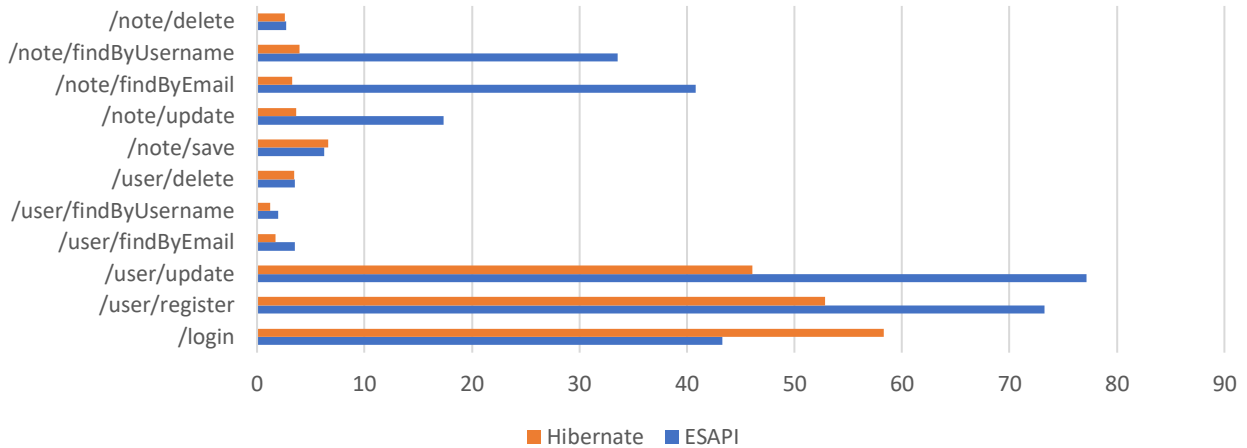
Tempi medi - SQL Injection (millisecondi)



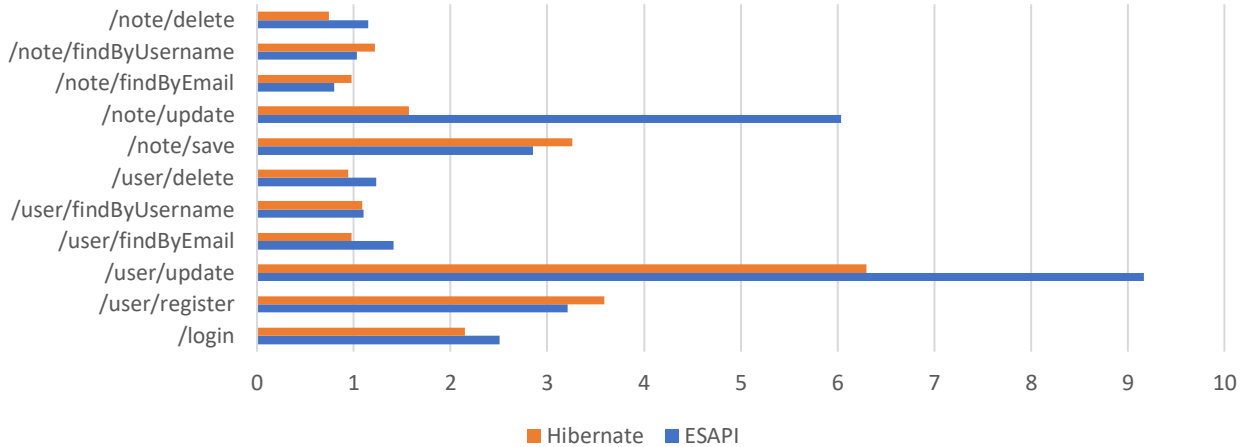
Tempi medi - Cross Site Scripting (millisecondi)



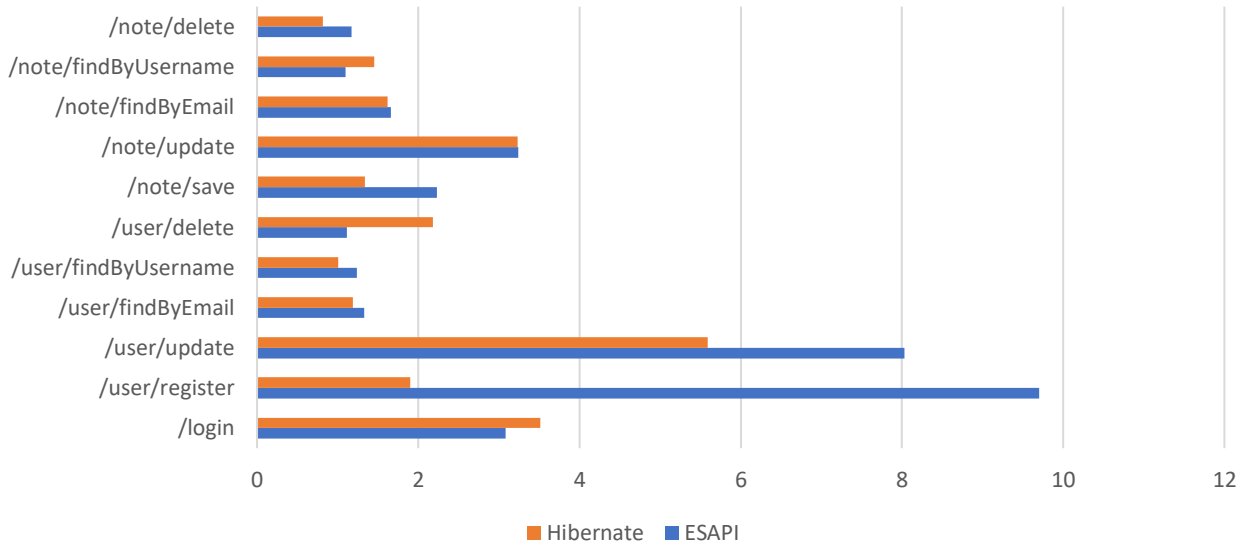
Deviazioni standard - Dati puliti (millisecondi)



Deviazioni standard - SQL Injection (millisecondi)



Deviazioni standard - Cross Site Scripting (millisecondi)



Differenze tempi medi e deviazioni standard

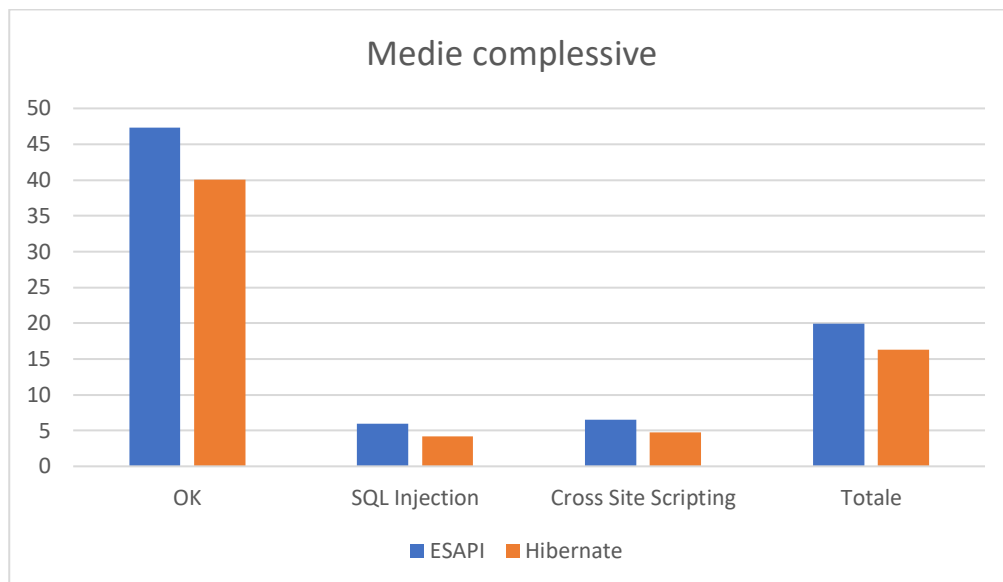
Unità: millisecondi HTTP Request	Differenze tempi medi OK - SQL Injection		Differenze deviazioni standard OK - SQL Injection	
	ESAPI	Hibernate	ESAPI	Hibernate
/login	89,87	106,286	40,8	56,18
/user/register	148,8	167,27	70,07	49,28
/user/update	110,87	93,24	68	39,77
/user/findByEmail	2,05	1,92	2,12	0,75
/user/findByUsername	1,38	0,69	0,86	0,12
/user/delete	5,36	6,53	2,26	2,5
/note/save	5,59	3,02	3,39	3,34
/note/update	12,91	2,93	11,34	2,07
/note/findByEmail	44,39	3,72	40,03	2,29
/note/findByUsername	31,23	3,5	32,5	2,74
/note/delete	2,54	5,44	1,56	1,83
Unità: millisecondi HTTP Request	Differenze tempi medi OK - Cross Site Scripting		Differenze deviazioni standard OK - Cross Site Scripting	
	ESAPI	Hibernate	ESAPI	Hibernate
/login	89,35	102,996	40,23	54,82
/user/register	141,77	169,23	63,58	50,97
/user/update	111,3	92,99	69,14	40,48
/user/findByEmail	3,07	0,61	2,2	0,54
/user/findByUsername	1,48	1,28	0,72	0,2
/user/delete	6,08	3,6	2,38	1,26
/note/save	5,24	4,93	4,01	5,26
/note/update	11,42	1,53	14,14	0,41
/note/findByEmail	43,84	2,68	39,17	1,65
/note/findByUsername	31,41	3,41	32,43	2,51
/note/delete	3,66	5,44	1,54	1,75
Unità: millisecondi HTTP Request	Differenze tempi medi tra OK e KO		Differenze deviazioni standard tra OK e KO	
	ESAPI	Hibernate	ESAPI	Hibernate
/login	89,61	104,641	40,515	55,5
/user/register	145,285	168,25	66,825	50,125
/user/update	111,085	93,115	68,57	40,125
/user/findByEmail	2,56	1,265	2,16	0,645
/user/findByUsername	1,43	0,985	0,79	0,16
/user/delete	5,72	5,065	2,32	1,88
/note/save	5,415	3,975	3,7	4,3
/note/update	12,165	2,23	12,74	1,24
/note/findByEmail	44,115	3,2	39,6	1,97
/note/findByUsername	31,32	3,455	32,465	2,625
/note/delete	3,1	5,44	1,55	1,79

Le performance presentano una tabella che mostra le differenze dei tempi medi e delle deviazioni standard in millisecondi. Queste differenze riguardano le richieste effettuate con i dati puliti nominati come «OK» e quelli con stringhe dannose

nominati come «KO». Hibernate sui tempi medi dimostra di essere più veloce rispetto ad ESAPI tranne per quanto riguarda la registrazione, login, cancellazione delle note. Mentre la cancellazione degli utenti solo nel caso del SQL Injection è più lento.

Media complessiva dei tempi medi di entrambi i progetti messi a confronto

Medie complessive		
Unità: millisecondi	ESAPI	Hibernate
OK	47,32272727	40,03236364
SQL Injection	5,96	4,164545455
Cross Site Scripting	6,539090909	4,696363636
Media complessiva	ESAPI	Hibernate
Totale	19,94060606	16,29775758



L'istogramma illustra invece i tempi medi complessivi e anche in questo caso, Hibernate è più veloce di ESAPI.

Robustezza:

Elemento	ESAPI	Hibernate
Gestione errori di validazione	Gestito senza interruzioni sulla Web Application	Gestito senza interruzioni sulla Web Application
Gestione errori di cattiva configurazione della validazione	Gestito con interruzioni sulla Web Application sui file properties e xml	Gestito con interruzioni sulla Web Application con le espressioni regolari e tipi di dati errati sui vincoli
Falle di sicurezza sulle validazioni e il loro impatto	Risolto il Cross Site Scripting sulle espressioni regolari dei metodi <code>getValidSafeHTML</code> e <code>isValidSafeHTML</code> .	Risolto il Cross Site Scripting sul vincolo di validazione <code>SafeHtml</code> che è stato deprecato.

Legenda tabella robustezza:

- Risolto: La falla di sicurezza è stata sistemata.
- Non risolto: La falla di sicurezza è ancora aperta.

La gestione degli errori viene gestita da entrambi quasi allo stesso modo. Sulle falle di sicurezza entrambe hanno avuto una falla ciascuno riguardante il Cross Site Scripting che sono state risolte. Ma Hibernate ha risolto la falla deprecando `SafeHtml` e ciò ha richiesto un vincolo di validazione personalizzato.

Conclusioni riassunte in una tabella:

Criteri di valutazione	Parametri di valutazione	Punteggio	
		ESAPI	Hibernate
Completezza	Validazione Centralizzata	1.5/3	0/3
	Validazione Distribuita	2/2	2/2
	Valutazione complessiva	3.5/5	2/5
Flessibilità	Personalizzazione validazione	3.5/4	4/4
	Documentazione	1.5/3	3/3
	Codice implementato	1.5/2	1.5/2
	Valutazione complessiva	6.5/9	8.5/9
Robustezza	Gestione errori	1.5/2	1.5/2
	Falle di sicurezza e il loro impatto	1.5/2	1/2
	Valutazione complessiva	4/5	2.5/5
Performance	Tempi medi complessivi	19,94ms	16,29ms

In conclusioni sono stati assegnati per ogni criterio di valutazione dei parametri e dei punteggi.

- 1 punto se la funzionalità del parametro è pienamente soddisfatta.
- 0.5 punto se la funzionalità è soddisfatta parzialmente.
- 0 punti se la funzionalità è assente.

Sulla validazione centralizzata ESAPI è più completo di Hibernate, tuttavia ha presentato qualche problema. Mentre su quella distribuita entrambe le librerie soddisfano le funzionalità. Sulla flessibilità Hibernate permette una

personalizzazione delle validazioni migliore rispetto a quella di ESAPI, anche la documentazione di Hibernate è migliore di quella fornita per la libreria ESAPI. Invece sulla robustezza entrambe gestiscono gli errori quasi allo stesso modo, sulle falle di sicurezza entrambe hanno avuto falle riguardanti il Cross Site Scripting che sono state risolte con la differenza che Hibernate ha dovuto deprecare il suo vincolo. Invece, sulle performance Hibernate è più veloce di ESAPI anche perché il progetto dedicato a Hibernate non ha la validazione centralizzata. Infine, per il reale utilizzo di ESAPI e Hibernate, bisogna valutare in base al progetto quale delle due utilizzare. Tenendo presente se il progetto viene fatto da zero oppure è già esistente.